

system will look around and choose the process with the shortest file. If there is a constant stream of processes with short files, the process with the huge file will never be allocated the printer. It will simply starve to death (be postponed indefinitely, even though it is not blocked).

Starvation can be avoided by using a first-come, first-served resource allocation policy. With this approach, the process waiting the longest gets served next. In due course of time, any given process will eventually become the oldest and thus get the needed resource.

It is worth mentioning that some people do not make a distinction between starvation and deadlock because in both cases there is no forward progress. Others feel that they are fundamentally different because a process could easily be programmed to try to do something  $n$  times and, if all of them failed, try something else. A blocked process does not have that choice.

## 6.8 RESEARCH ON DEADLOCKS

If ever there was a subject that was investigated mercilessly during the early days of operating systems (1960s and 1970s), it was deadlocks. The reason is that deadlock detection is a nice little graph-theory problem that one mathematically inclined graduate student could get his jaws around and chew on for 4 years. Many algorithms were devised, each one more exotic and less practical than the previous one. Most of that work has died out. Still, a few papers are still being published on deadlocks.

Recent work on deadlocks includes new approaches to diagnose concurrency problems such as deadlocks. The challenge here is to reproduce the schedules, or “thread interleavings,” that lead to the deadlock, livelock, and similar conditions. Unfortunately, recording in detail the scheduling decisions in production systems is too expensive. Instead, researchers are looking for ways to record the interleavings at a coarser granularity while guaranteeing reproducibility of the deadlock or livelock problem (Kasikci et al., 2017).

Marino et al. (2013), on the other hand, use concurrency control to make sure that deadlocks cannot occur in the first place. In contrast, Duo et al. (2020) use formal modeling to derive constraints on the scheduling to ensure deadlocks will not happen.

Solutions for deadlock detection are not limited to a single system. For instance, Hu et al. (2017) give a method that prevents deadlocks due to the use of Remote DMA (RDMA) in data centers. RDMA is just like regular DMA as discussed in Chap. 5, except that the DMA transfer is now initiated from a remote machine across the network. In a specific mode, known as priority flow control, the RDMA transfer requires the (exclusive) reservation of RDMA buffers in the intermediate nodes in the network. It does so to make sure there can be no packet drops due to buffers overflowing. However, these buffers are just the sort of limited

resources that we discussed in this chapter. Suppose two hosts both want to perform an RDMA transfer and both have to use buffers on intermediate nodes  $N_1$  and  $N_2$ . If one of the hosts reserves the last buffer on  $N_1$  and the other the last buffer on  $N_2$ , neither can make progress. By ensuring that the packets from different flows end up in different buffers, Hu et al. (2017) show that deadlocks are not possible.

Another research direction is to try and detect deadlocks. For instance, Pyla and Varadarajan (2012) present a deadlock detection system that associates memory updates with one or more locks guarding the updates and does not make the updates globally visible until all locks that protect the updates are released. All memory updates in a critical region are then performed atomically. By checking at the acquisition of the locks, it is possible to detect deadlocks early and start the recovery procedure. Recovery from a deadlock consists simply of picking one of the locks and discarding all pending memory updates associated with it. The work by Cai and Chan (2012) presents a new dynamic deadlock detection scheme that iteratively prunes lock dependencies that have no incoming or outgoing edges.

Finally, there is a huge amount of theoretical work on distributed deadlock detection. However, we will not consider it here because (1) it is outside the scope of this book and (2) none of it is even remotely practical in real systems. Its main function seems to be keeping otherwise unemployed graph theorists off the streets.

## 6.9 SUMMARY

Deadlock is a potential problem in any operating system. It occurs when all the members of a set of processes are blocked waiting for an event that only other members of the same set can cause. This situation causes all the processes to wait forever. Commonly the event that the processes are waiting for is the release of some resource held by another member of the set. Another situation in which deadlock is possible is when a set of communicating processes are all waiting for a message and the communication channel is empty and no timeouts are pending.

Resource deadlock can be avoided by keeping track of which states are safe and which are unsafe. A safe state is one in which there exists a sequence of events that guarantee that all processes can finish. An unsafe state has no such guarantee. The banker's algorithm avoids deadlock by not granting a request if that request will put the system in an unsafe state.

Resource deadlock can be structurally prevented by building the system in such a way that it can never occur by design. For example, by allowing a process to hold only one resource at any instant, the circular wait condition required for deadlock is broken. Resource deadlock can also be prevented by numbering all the resources and making processes request them in strictly increasing order.

Resource deadlock is not the only kind of deadlock. Communication deadlock is also a potential problem in some systems although it can often be handled by setting appropriate timeouts.